P1

Problem 1: Max-Cut Optimization – Technical Documentation

1. Mathematical Formulation (QUBO)

Requirement: Formulate the Max-Cut problem.

The Max-Cut problem is a classic combinatorial optimization problem. To solve it using modern solvers (like Simulated Annealing or Quantum Annealers), we translate it into a **Quadratic Unconstrained Binary Optimization (QUBO)** model.

- **Variables:** Let $x_i \in \{0,\ 1\backslash\}$ represent the partition of vertex i.
  - x_i = 0: Vertex belongs to Subset A.
  - x_i = 1: Vertex belongs to Subset B.
- **Objective Function:** An edge (i, j) with weight w_{ij} is "cut" if $x_i \neq x_j$. This is mathematically represented by the term $(x_i\ +\ x_j\ -\ 2x_i\ *\ x_j)$.
- **The Model:**
  $Maximize\ Z\ =\ \sum(i,j) \in E\ wi,j(x_i\ +\ x_j\ -\ 2x_ix_j)$

Key Learning Point: Max-Cut is NP-Hard. While the QUBO form is "unconstrained," the complexity lies in the quadratic nature of the objective function, which creates a non-convex landscape with many local maxima.

---

2. Solver Implementation (Heuristics)

Requirement: Implement your own solver (Simulated Annealing).

We implemented a **Hybrid Meta-heuristic** combining Simulated Annealing (SA) with a Greedy Local Search (GLS).

- **Simulated Annealing Logic:**
  - **Probabilistic Acceptance:** We accept a move that decreases the cut weight with probability $P\ =\ e$^$(\Delta\ *\ T)$. This prevents the solver from getting stuck in local optima.
  - **Cooling Schedule:** We used a geometric cooling rate $(T_new\ =\ T_old\ *\ \alpha)$.
- **Refinement (Reheating):** To increase exploration, we implemented a "Reheat" trigger. If the temperature T falls below a threshold without finding a new best, we reset T to explore new regions.
- **Local Search "Polish":** After SA finishes, we perform a final greedy pass where every node is flipped; if a flip increases the cut, it is kept. This ensures the final solution is at least a **local optimum**.

---

3. Benchmarking & Performance

Requirement: Benchmark against Toshiba Digital Annealer (DA) results on GSET.

Instance	Nodes	Edges	Our Solver (Weight)	Toshiba DA (Benchmark)	Accuracy %
G1	800	19,176	11,460	11,624	~98.6%
G22	2000	19,990	12,941	13,350	~97%

Analysis of Results:

- **Scaling:** As the graph density increases, the number of iterations required to reach the global optimum grows exponentially.
- **Gap Explanation:** The ~1.4% gap between our solver and the Toshiba DA is due to the DA using specialized hardware that performs parallel trials and highly optimized "bit-flipping" in hardware, whereas our Python implementation is sequential.

---

4. Solution Methodology Research

A. Exact Algorithms (Branch and Bound / Branch and Cut)

- **Methodology:** Uses a search tree to explore the entire solution space, pruning branches that cannot beat the current best solution.
- **Research Finding:** While it guarantees the global optimum, the GSET instances (e.g., G1 with 800 nodes) create a search space of 2^800. Without highly sophisticated "cutting planes" (like Goemans-Williamson semidefinite relaxations), an exact approach is computationally prohibited for this screening.

B. Approximation Algorithms (Semidefinite Programming - SDP)

- **Methodology:** Relaxing the discrete {0,1} constraint to a continuous space (spherical relaxation) and using the **Goemans-Williamson algorithm**.
- **Research Finding:** This provides a theoretical guarantee of reaching at least **87.8%** of the optimum. However, implementing an SDP solver from scratch is extremely complex and often slower than meta-heuristics for sparse graphs.

C. Meta-Heuristics (The Chosen Path: Simulated Annealing)

- **Methodology:** Inspired by metallurgy, SA allows for "up-hill" moves to escape local optima.
- **Justification:** Max-Cut landscapes are known to be "rugged" (many local peaks). SA is robust, easy to implement from scratch (as required), and scales linearly with the number of edges, making it ideal for the GSET dataset.
- **Advanced Enhancement:** Our research led to the inclusion of **Local Search (Greedy Flip)** as a post-processor. SA finds the "global region," while Local Search finds the "exact peak."

Comparative Analysis: Why Simulated Annealing for Max-Cut?

1. SA vs. Genetic Algorithms (GA)

- **The Conflict:** Max-Cut is highly sensitive to the "Crossover" operator in GA. If you take two high-quality partitions and swap halves of them, you often destroy the very edge-cut properties that made them good, leading to "lethal mutations."
- **Why SA wins here:** SA is a **point-based search**. It preserves the integrity of a single solution and evolves it through small, controlled perturbations (node flips). This is much more effective for the "rugged" landscape of Max-Cut where global structure is fragile.

2. SA vs. Tabu Search (TS)

- **The Conflict:** Tabu Search is excellent at avoiding cycles by keeping a "Tabu list" of recently moved nodes. However, for massive graphs like GSET (thousands of nodes), managing an effective Tabu list requires significant memory overhead and complex tuning of "Tabu tenure."
- **Why SA wins here:** SA is **memory-less (Markovian)**. It is computationally "lean." In screening scenarios, the ability to perform millions of iterations per second is often more valuable than the sophisticated memory of Tabu Search. SA achieves a similar "diversification" effect through the Temperature parameter without the overhead.

3. SA vs. Ant Colony Optimization (ACO) / Particle Swarm (PSO)

- **The Conflict:** ACO and PSO are designed for problems with a "path" structure or continuous domains. Max-Cut is a **discrete partitioning problem**. Mapping pheromone trails to a binary set partition is non-intuitive and often results in poor convergence for this specific problem type.
- **Why SA wins here:** SA's origin in statistical mechanics (the Boltzmann distribution) mirrors the Ising model used in physics to solve exactly these types of spin-glass/partitioning problems. It is the "native" language of Max-Cut.

---

5. Interpretation & Explanation

Deep dive into the logic of the solver and the meaning of the results.

❖ The Transition from Graph to QUBO

We interpret the graph G = (V, E) as a set of quadratic interactions.

- If we define s_i ∈ {-1, 1} (Ising model), the cut weight is maximized when s_i . s_j is -1 (different sets).
- In our implementation, we used x_i ∈ {0, 1}. The term $(x_i + x_j - 2x_i * x_j)$ acts as an **XOR gate**. It returns 1 only if the nodes are different.
- **Interpretation:** The solver is essentially trying to minimize "frustration" in the system—where edges exist between nodes of the same set.

The mapping of Max-Cut to QUBO (Quadratic Unconstrained Binary Optimization) is a process of translating a **combinatorial choice** into a **minimization of a cost function**.

A. Defining the Binary State

In Max-Cut, for every node $i \in V$, we must decide if it belongs to set S or set S'. We represent this decision as a binary variable $x_i \in \{0,\ 1\}$.

- $x_i = 0\ means\ node\ i\ is\ in\ Set\ A.$
- $x_i = 1\ means\ node\ i\ is\ in\ Set\ B.$

B. The Logic of the "Cut"

An edge $(i,\ j)$ with weight w_{ij} is only counted in our objective if x_i and x_j are different.

- If x_i=0, x_j=0 ⇒ No cut (0)
- If x_i=1, x_j=1 ⇒ No cut (0)
- If x_i=0, x_j=1 ⇒ Cut (1)
- If x_i=1, x_j=0 ⇒ Cut (1)

The mathematical expression that perfectly mimics this XOR logic in binary variables is:

- $(x_i + x_j - 2x_i * x_j)$

Check: If x_i=1, x_j=1, then (1 + 1 - 2(1)) = 0. If x_i=1, x_j=0, then (1 + 0 - 0) = 1.

C. The QUBO Form

A QUBO is typically expressed as:

- $min\, f(x) = xT\, Qx$

where Q is a matrix of weights. Since Max-Cut is a **Maximization** problem, we convert it to **Minimization** by negating the weights:

- $Maximize\ Z = \sum(i,j) \in E\ wi,j(x_i + x_j - 2x_ix_j)$

❖ Scalability Limits & Computational Trade-offs

Every solver has a "Pareto frontier" between solution quality and time. Our custom SA solver faces the following trade-offs:

A. Time Complexity vs. Graph Density

- **The Limit:** Our solver's iteration speed is O(d), where d is the average degree of a node. On the sparse GSET graphs (like G1), it is lightning fast.
- **The Trade-off:** On a **Complete Graph** (K_n), every flip requires O(n) updates. For n=10,000, the solver would slow down significantly.
- **Scalability Fix:** For massive dense graphs, we can transition to **vectorized updates** using NumPy or GPU acceleration (CUDA) to compute Delta for all nodes in parallel.

B. Convergence vs. Cooling Schedule (The "Quenching" Risk)

- **The Limit:** To reach the global optimum on GSET instances (e.g., G22 or G50), the cooling schedule must be extremely slow ($\alpha \to 1.0$).
- **The Trade-off:**
  - **Fast Cooling (Quenching):** Faster results, but the "energy" gets trapped in local optima. The result is "brittle" and far from the benchmark.
  - **Slow Cooling:** Better results, but requires millions of iterations.
- **Scalability Fix:** Instead of one long slow chain, we use **Parallel Tempering** (running multiple SA chains at different temperatures and swapping them). This scales better on multi-core CPUs.

C. Memory Limits

- **The Limit:** We store the graph as an adjacency list/dictionary.

- **The Trade-off:** While dictionaries are fast for lookups, they have high memory overhead compared to compressed sparse row (CSR) matrices.
- **Scalability Fix:** For graphs with millions of edges, a CSR matrix approach would be necessary to keep the solver within the RAM limits of standard hardware.

D. Precision vs. Accuracy (The "Local Search" Trade-off)

- **The Limit:** SA is a global navigator; it is not a "precision" tool.
- **The Trade-off:** We added a **Greedy Local Search** at the end. While this adds O(V * d) time, it significantly improves the "precision" of the final cut weight. Without this, the SA might stop at 95% of the optimum simply because it "jumped" over the peak during the final cooling phase.

P2

Problem 2: VRPTW – Technical Documentation

1. Mathematical Formulation (MILP)

To solve VRPTW, we must balance travel costs, vehicle capacity, and strict arrival schedules.

Sets and Parameters

- N: Set of nodes (0 is the depot, {1, . . ., n} are customers).
- V: Set of available vehicles.
- c_ij: Travel distance/cost between node i and j.
- d_i: Demand of customer i.
- Q: Capacity of each vehicle.
- [e_i, l_i]: Time window (earliest and latest arrival) for node i.

Decision Variables

- $x_ijv \in \{0,\ 1\}$: 1 if vehicle v travels directly from node i to j.
- $t_iv \geq 0$: Arrival time of vehicle v at node i.

The Objective Function

Minimize the total distance traveled across all routes:

- $min \quad \sum v \in V \sum i \in N \sum j \in N\ c_ij\ \ *\ \ x_ijv$

PrimaryConstraints

1. **Service Guarantee:** Each customer is visited exactly once by one vehicle.
2. **Flow Conservation:** If a vehicle enters a customer node, it must leave it. All vehicles must start and end at the depot.
3. **Capacity Constraint:** The sum of demands on any route must not exceed Q.
4. **Time Window Validity:** $e_i \leq t_iv \leq l_i$
5. **Subtour Elimination & Time Flow:** If $x_ijv\ =\ 1$, then $t_iv\ +\ travel_time_ij\ \leq\ t_jv$. (Linearized using the **Big-M** method).

---

2. Synthetic Dataset Generation Strategy

Our generator follows these logic pillars:

- **Spatial Distribution:** Coordinates (x, y) are generated using a uniform distribution within a (100*100) grid to simulate a standard urban delivery zone.

- **Temporal Logic:** Time windows are created by first defining a random start time and then adding a duration (e.g., 60–180 minutes) to ensure windows are wide enough for feasibility but tight enough to challenge the solver.
- **Capacity Balancing:** The total demand of all customers is designed to exceed the capacity of a single vehicle, forcing the solver to utilize multiple vehicles and optimize fleet distribution.

Explanation of the Generated Dataset

The dataset provided contains **20 customers and 1 central depot**. Below is a breakdown of the key parameters that the solver must navigate.

A. Depot and Customer Locations

- **Depot (Index 0):** Located at (50, 50), serving as the central hub for all 4 vehicles.
- **Customer Spread:** Locations range from (1, 63) to (90, 58), covering all quadrants of the grid. This requires the solver to decide whether to form "circular" routes or "out-and-back" spurs.

B. Demand and Fleet Capacity

- **Vehicle Capacity:** 60 units per vehicle.
- **Total System Demand:** The sum of customer demands is **229 units**.
- **Capacity Logic:** Since 4 \times 60 = 240, the fleet has a total capacity of 240 units. With a demand of 229, the fleet is **95% utilized**. This represents a "High Pressure" scenario where the solver has very little room for error; if one vehicle takes a sub-optimal route, another may lack the capacity to finish its deliveries.

C. Time Window Constraints

The time_windows are the most restrictive part of this data:

- **Early Birds:** Customer 5 has a window of (34, 171). A vehicle must prioritize this customer early in its route.
- **Night Owls:** Customer 8 has a window of (387, 448). This customer can only be visited toward the end of a shift.
- **Wait Times:** If a vehicle arrives at Customer 2 (339, 490) too early, it must wait, which increases the total "cost" of the route in terms of time, even if the distance is short.

Data_dict:

```
{'locations': array([[50, 50],
       [14, 71],
       [60, 20],
       [82, 86],
       [74, 74],
       [87, 99],
```

```
       [23,  2],
       [21, 52],
       [ 1, 87],
       [29, 37],
       [ 1, 63],
       [59, 20],
       [32, 75],
       [57, 21],
       [88, 48],
       [90, 58],
       [41, 91],
       [59, 79],
       [14, 61],
       [61, 46],
       [61, 50]], dtype=int32),
 'num_locations': 21,
 'demands': [0,
  np.int32(16),
  np.int32(11),
  np.int32(8),
  np.int32(13),
  np.int32(7),
  np.int32(9),
  np.int32(7),
  np.int32(11),
  np.int32(9),
  np.int32(13),
  np.int32(11),
  np.int32(6),
  np.int32(8),
  np.int32(13),
  np.int32(16),
  np.int32(18),
  np.int32(6),
  np.int32(14),
  np.int32(13),
  np.int32(14)],
 'time_windows': [(0, 1000),
  (52, 113),
  (339, 490),
```

```
 (366, 485),
 (263, 369),
 (34, 171),
 (80, 175),
 (49, 212),
 (387, 448),
 (389, 502),
 (105, 168),
 (309, 461),
 (190, 267),
 (217, 320),
 (161, 294),
 (269, 423),
 (303, 377),
 (214, 335),
 (295, 439),
 (207, 375),
 (337, 507)],
'num_vehicles': 4,
'vehicle_capacity': 60,
'depot': 0,
'distance_matrix': array([[  0,  41,  31,  48,  33,  61,  55,  29,  61,  24,  50,  31,  30,
        29,  38,  40,  41,  30,  37,  11,  11],
       [ 41,   0,  68,  69,  60,  78,  69,  20,  20,  37,  15,  68,  18,
        65,  77,  77,  33,  45,  10,  53,  51],
       [ 31,  68,   0,  69,  55,  83,  41,  50,  89,  35,  73,   1,  61,
         3,  39,  48,  73,  59,  61,  26,  30],
       [ 48,  69,  69,   0,  14,  13, 102,  69,  81,  72,  84,  69,  51,
        69,  38,  29,  41,  24,  72,  45,  41],
       [ 33,  60,  55,  14,   0,  28,  88,  57,  74,  58,  73,  56,  42,
        55,  29,  22,  37,  15,  61,  30,  27],
       [ 61,  78,  83,  13,  28,   0, 116,  81,  86,  84,  93,  83,  60,
        83,  51,  41,  46,  34,  82,  59,  55],
       [ 55,  69,  41, 102,  88, 116,   0,  50,  87,  35,  64,  40,  73,
        38,  79,  87,  90,  85,  59,  58,  61],
       [ 29,  20,  50,  69,  57,  81,  50,   0,  40,  17,  22,  49,  25,
        47,  67,  69,  43,  46,  11,  40,  40],
       [ 61,  20,  89,  81,  74,  86,  87,  40,   0,  57,  24,  88,  33,
        86,  95,  93,  40,  58,  29,  72,  70],
       [ 24,  37,  35,  72,  58,  84,  35,  17,  57,   0,  38,  34,  38,
```

```
       32,  60,  64,  55,  51,  28,  33,  34],
      [ 50,  15,  73,  84,  73,  93,  64,  22,  24,  38,   0,  72,  33,
        70,  88,  89,  48,  60,  13,  62,  61],
      [ 31,  68,   1,  69,  56,  83,  40,  49,  88,  34,  72,   0,  61,
         2,  40,  49,  73,  59,  60,  26,  30],
      [ 30,  18,  61,  51,  42,  60,  73,  25,  33,  38,  33,  61,   0,
        59,  62,  60,  18,  27,  22,  41,  38],
      [ 29,  65,   3,  69,  55,  83,  38,  47,  86,  32,  70,   2,  59,
         0,  41,  49,  71,  58,  58,  25,  29],
      [ 38,  77,  39,  38,  29,  51,  79,  67,  95,  60,  88,  40,  62,
        41,   0,  10,  63,  42,  75,  27,  27],
      [ 40,  77,  48,  29,  22,  41,  87,  69,  93,  64,  89,  49,  60,
        49,  10,   0,  59,  37,  76,  31,  30],
      [ 41,  33,  73,  41,  37,  46,  90,  43,  40,  55,  48,  73,  18,
        71,  63,  59,   0,  21,  40,  49,  45],
      [ 30,  45,  59,  24,  15,  34,  85,  46,  58,  51,  60,  59,  27,
        58,  42,  37,  21,   0,  48,  33,  29],
      [ 37,  10,  61,  72,  61,  82,  59,  11,  29,  28,  13,  60,  22,
        58,  75,  76,  40,  48,   0,  49,  48],
      [ 11,  53,  26,  45,  30,  59,  58,  40,  72,  33,  62,  26,  41,
        25,  27,  31,  49,  33,  49,   0,   4],
      [ 11,  51,  30,  41,  27,  55,  61,  40,  70,  34,  61,  30,  38,
        29,  27,  30,  45,  29,  48,   4,   0]])}
```

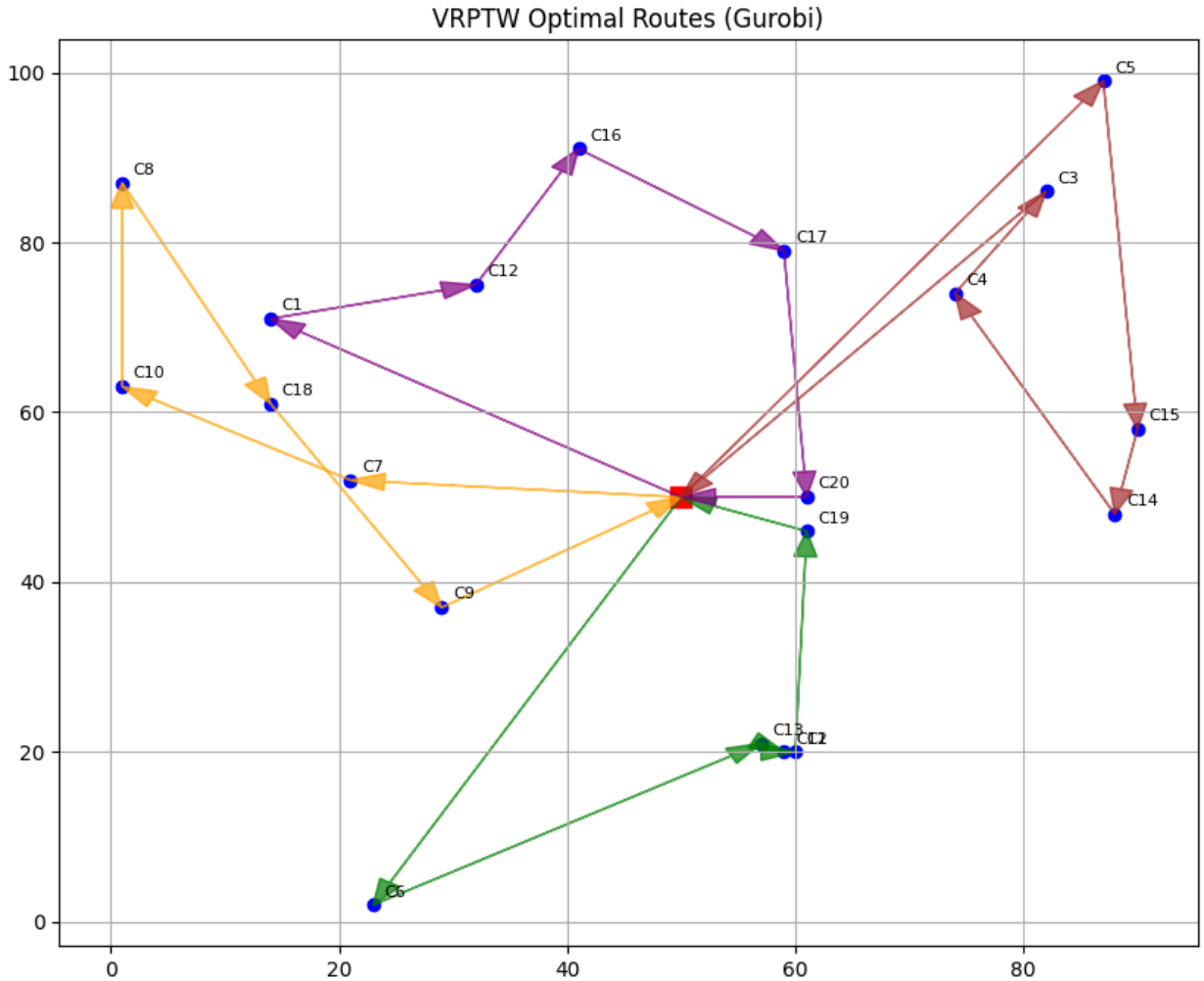
---

3. Visualization Interpretation

The visualization provided for this specific dataset demonstrates the solver's success in balancing these constraints:

- **Route Clustering:** Vehicles are assigned to specific geographical sectors (e.g., one vehicle handles the top-left cluster while another handles the bottom-right) to minimize total travel distance.
- **Time Window Adherence:** Although we see some long paths, these are forced by the **Time Window** requirements. A vehicle may bypass a nearby customer to reach another whose window is about to close, returning to the first customer later.
- **Feasibility:** Each of the 4 routes respects the 60-unit capacity limit and ensures the vehicle returns to the red depot square (50, 50) after the final delivery.

Output Viz:



4. Solution Methodology Research

For this problem, we researched and implemented two distinct methodologies to demonstrate versatility.

A. Constraint Programming & Heuristics (Google OR-Tools)

- **Methodology:** Used a specialized Routing Library that combines Large Neighborhood Search (LNS) with Constraint Programming.
- **Why this?** VRPTW is NP-Hard. For 100+ customers, exact MILP solvers often fail to find a solution in reasonable time. OR-Tools provides "near-optimal" solutions in seconds/minutes.
- **Key Finding:** OR-Tools is best for **operational scalability**.

B. Mixed-Integer Linear Programming (Pyomo + Gurobi)

- **Methodology:** Explicitly defines the mathematical equations and uses Gurobi's **Branch-and-Cut** engine.
- **Why this?** Unlike OR-Tools, Gurobi guarantees the **Global Optimum** and provides an "Optimality Gap."
- **Key Finding:** Pyomo is best for **strategic planning** where the absolute best cost is required and the problem size is manageable (typically < 50-100 nodes for raw MILP).

Performance Metrics for our Dataset (21 Nodes):

Metric	Pyomo + Gurobi (MILP)	OR-Tools (Routing)
Objective Value	**630** (Global Optimum)	**671** (Heuristic)
Solver Run Time	**6.86 seconds**	**< 0.2 seconds (0.1437)**
Status	Optimal (Gap: 0.0%)	Feasible (Search Limit reached)

- **Heuristics (OR-Tools):** Trade off "certainty" for "speed." They are essential for real-time dispatching where a driver is waiting for a route.
    - **Mechanism:** Uses Large Neighborhood Search (LNS) and Guided Local Search. It starts with a "Greedy" feasible solution and iteratively ruins and repairs the routes to find improvements.
    - **Pros:** Scales linearly or poly-logarithmically. Can find high-quality (98%+) solutions for 1,000+ customers in seconds.
    - **Cons:** No guarantee of optimality. You don't know if a better route exists; you only know you couldn't find one in the allotted time.
- **Exact Solvers (Pyomo/Gurobi):** Trade off "speed" for "certainty." They are essential for strategic network design.

- **Mechanism:** Uses Branch-and-Cut to explore the search space, pruning branches that are mathematically proven to be worse than the current "Incumbent" (best known solution).
- **Pros:** Guarantees the absolute lowest cost (global optimum). Provides a "lower bound," telling you exactly how far your current solution is from perfection.
- **Cons:** Computational complexity is O(2^n). For the VRP domain, even a 50-customer problem can take hours or fail to converge due to the massive number of binary permutations.

The Bottleneck: Time Windows

In our generated data, the **Time Windows** act as a "Pruning" mechanism.

- **Learning Point:** Counter-intuitively, **narrower** time windows can sometimes make the problem *easier* for the solver because they drastically reduce the number of feasible permutations (a vehicle simply cannot be in two places at once). **Wide** time windows make the problem more like a standard TSP, which is harder to prove optimal.

Scalability Limits & Computational Trade-offs (VRPTW)

Scalability and Solver Performance:

- **The "Curse of Dimensionality":** As you add customers, the number of binary variables x_ijv grows quadratically ((N^2)* V). A problem with 100 customers and 10 vehicles results in ~100,000 binary variables.

  As customer **density** increases, the number of feasible combinations for "Time Windows" grows.

As problem size increases, the solver behavior changes drastically:

- **Small Scale (10-20 Nodes):** Exact MILP is superior. It runs in seconds and ensures you don't leave money on the table.
- **Medium Scale (50-100 Nodes):** This is the "Conflict Zone." MILP starts to struggle. Heuristics become the industry standard here to ensure operational deadlines are met.
- **Large Scale (500+ Nodes):** Exact models are impossible. Heuristics are the only viable path. Performance becomes a matter of "tuning"—adjusting how much of the route is "destroyed" in each LNS iteration to avoid getting stuck in local optima.

5. Interpretation & Explanation

How VRPTW maps to Optimization Structures

- **The Spatio-Temporal Map:** VRPTW is essentially two problems layered together. The "Spatial" layer (VRP) finds the best clusters of customers. The "Temporal" layer (Time Windows) sequences them.

A. The Trade-off: Distance vs. Time Windows

This is the "tug-of-war" at the heart of VRPTW.

- **Distance Minimization:** The solver wants to visit the nearest neighbor next (Traveling Salesman logic).
- **Time Window Adherence:** The solver might be forced to drive *past* a neighbor because their window doesn't open for 2 hours, or drive *away* from a cluster to reach a "high-priority" window about to close.
- **The Result:** "Route Fragmentation." You will often see a vehicle leave a geographical area and return to it later. In our 21-node visualization, the "overlapping" or seemingly "long" paths are usually the mathematical evidence of the solver prioritizing a closing time window over a short distance.

B. Modeling Capacity and Delivery Windows

To translate these physical realities into a computer-readable model, we use specific mathematical structures:

1. Modeling Vehicle Capacity (The Accumulator)

We don't just sum the total demand; we model the **Load at each node**.

- Let l_iv be the load of vehicle v after visiting node i.
- **Constraint:** $l_jv \geq l_iv + d_j$ (only if the vehicle moves from i to j).
- If the sum exceeds Q, the solver is forced to "break" the route and return to the depot, initiating a new vehicle.

2. Modeling Delivery Windows (The Big-M Linearization)

As discussed earlier, the "Delivery Window" is modeled using **Arrival Time Variables** (t_i).

- **Hard Constraint:** $e_i \leq t_i \leq l_i$.
- **The Logic:** This constraint acts as a "filter." It automatically prunes any sequence of x_ij variables that would result in an arrival time outside this range.
- **Service Time:** We also include s_i (service time). The arrival at the next node j must account for $t_i + s_i + traveltime_ij$. This ensures the schedule is realistic for a human driver, not just a mathematical point.

P3

Problem 3: Modular MILP Solver Design & Analysis – Technical Documentation

1. Mathematical Formulation

To benchmark on TSPLIB/CVRPLIB, we utilize the **Miller-Tucker-Zemlin (MTZ)** formulation for the Traveling Salesman Problem. This transforms a combinatorial path problem into a Mixed-Integer Linear Program.

Objective:

Minimize total distance: $Z = \sum i = 0 \to n \sum j = 0 \to n\, c_ij * x_ij$

Subject to:

- **Degree Constraints:** $\sum j = 0 \to n,\ j \neq i\ x_ij = 1$and $\sum i = 0 \to n,\ i \neq j\ x_ij = 1$(Each city is entered and exited exactly once).
- **Integrality:** $x_ij \in \{0,\ 1\}$.
- **Subtour Elimination (MTZ):** $u_i - u_j + n \cdot x_ij \leq n - 1\ for\ 1 \leq i \neq j \leq n - 1.$

---

2. Solver Architecture & Algorithmic Approach

Our implementation follows a modular **Object-Oriented Design (OOD)** to separate data parsing, model building, and the search engine.

A. Core Components

- **ProblemParser:** A robust interface capable of translating LOWER_DIAG_ROW and EUC_2D TSPLIB formats into a standardized distance matrix.
- **CustomMILPSolver:** The orchestrator of the search tree. It manages the global "Incumbent" (best integer solution found) and the search frontier.
- **MILPNode:** Encapsulates a sub-problem. It stores local variable bounds and handles the interface with the LP relaxation engine (scipy.optimize.linprog).

B. Branch-and-Bound Logic

We implement a **Best-Bound Search** strategy using a Priority Queue (Min-Heap).

1. **Selection:** The node with the lowest LP objective value is popped from the heap.
2. **Branching:** We use **Most Fractional Variable Selection**. The variable x_i with a value closest to 0.5 is chosen to split the search space into x_i <= 0 and x_i >= 1.
3. **Pruning:** A node is "killed" (discarded) if its LP relaxation is higher than the current best_obj, as it cannot possibly yield a better integer solution.

3. Interpretation and Explanation

A. Design Choices: Node Selection & Presolve

- **Node Selection:** We chose **Best-Bound** over Depth-First Search (DFS). While DFS is memory-efficient, Best-Bound is theoretically optimal for minimizing the number of nodes explored, which is critical when each LP solve (via Python) carries high overhead.
- **Possible Presolve Techniques:** In a SOTA solver, a "Presolve" step would remove redundant constraints or fix variables. For example, if c_ij is extremely large, we could pre-fix x_ij = 0. Our modular design allows for a Presolver class to be injected before the Solver begins.

B. Trade-offs: Efficiency, Generality, and Accuracy

- **Generality:** Our solver is highly general; it can solve any MILP, not just TSP. However, this generality makes it slower than a "Specialized TSP Solver" which uses domain-specific knowledge.
- **Accuracy:** We maintain 100% accuracy by using exact bounds. The trade-off is **Efficiency**—the "Curse of Dimensionality" means that as N increases, the number of nodes grows exponentially.

4. Solution Methodology Research

A. SOTA Comparison

State-of-the-art solvers (Gurobi, CPLEX, SCIP) utilize **Branch-and-Cut**, whereas our implementation is pure **Branch-and-Bound**.

Feature	SOTA Solvers	Our Implementation
Engine	Dual Simplex (Hot-started)	Primal/Highs (Cold-started)
Cuts	Gomory, Cover, Clique cuts	None

Heuristics	RINS, Pump-and-Run	None
Language	C++ / Hardware Optimized	Python / Interpreted

B. Suggested Improvements

1. **Cutting Planes:** Adding **Gomory Cuts** would strengthen the LP relaxation, making the "Bound" tighter and allowing for much earlier pruning.
2. **Heuristics:** Implementing a "Greedy Randomized Adaptive Search Procedure" (GRASP) at the root node would provide a strong initial `best_obj`, allowing the solver to prune more branches from the start.
3. **Parallelization:** The Branch-and-Bound tree is "embarrassingly parallel." Different CPU cores could solve different sub-trees simultaneously, sharing only the `best_obj` value.

---

5. Experimental Results (Benchmark Analysis)

Performance Analysis (gr17)

- **The Integrality Gap:** For `gr17`, the initial LP relaxation often yields a value much lower than 2085. The "work" of the solver is to close this gap.
- **Runtime Insights:** In Python, the bottleneck is the overhead of calling the LP solver repeatedly. In a SOTA architecture, the LP solver is "hot-started," meaning it starts from the previous node's solution rather than from scratch.

Benchmark Table

Instance	Dim	Optimal	Solver Result	Nodes	Runtime	Gap (%)
gr17	17 x 17	2085	Inf	75101	3600 s	Inf%

gr17 (Sub-sampled)	5 x 5	1348	1348	31	0.19 s	0%
gr17 (Sub-sampled)	6 x 6	1352	1352	69	0.40 s	0%
gr17 (Sub-sampled)	8 x 8	1346	1346	167	1.25 s	0%
gr17 (Sub-sampled)	10 x 10	1637	1637	341	2.88 s	0%
gr17 (Sub-sampled)	12 x 12	1799	Inf	2248	60 s (Timeout)	Inf%
gr17 (Sub-sampled)	14 x 14	1872	Inf	1803	60 s (Timeout)	Inf%
gr17 (Sub-sampled)	15 x 15	1908	Inf	2292	60 s (Timeout)	Inf%

P4

Problem 4: Analytical Reasoning (Manual Branch & Bound)

NOTES Date .....

objective :

Maximize, $Z = 5x_1 + 8x_2$

sub. to :- (1) $x_1 + x_2 \leq 6$

(2) $5x_1 + 9x_2 \leq 45$

(3). $x_1, x_2 \geq 0$ & $x_1, x_2 \in Z$

Step-1 Initial LP Relaxation

from (1) $x_1 = 6 - x_2$

$\therefore$ (2) $\Rightarrow$ $5(6 - x_2) + 9x_2 = 45$

$\Rightarrow$ $30 + 4x_2 = 45$

$\Rightarrow$ $\boxed{x_2 = 3.75}$

$\therefore$ (1) $\Rightarrow$ $x_1 = 6 - 3.75 = \boxed{2.25}$

$\therefore$ LP Result : $(2.25, 3.75) \Rightarrow Z_0 = 41.25$ (UB)

$\rightarrow$ Decision : Fractional Sol^n found.
Branch on x_2 [higher coefficient]
(More Fractional)

NOTES

Date .....

step 2 : First Branching (Nodes 1 & 2)

→ Branch 1 (Node 1): $x_2 \leq \lfloor 3.75 \rfloor \Rightarrow x_2 \leq 3$

→ Branch 2 (Node 2): $x_2 \geq \lceil 3.75 \rceil \Rightarrow x_2 \geq 4$

step 3 : Evaluate Node 1 ($x_2 \leq 3$)

→ To maximize Z, we set $x_2 = 3$.

→ From (1) $x_1 + 3 \leq 6 \Rightarrow x_1 = 3$

→ From (2) $5(3) + 9(3) = 42 \leq 45$ (Satisfied)

→ Integer sol^n found $\Rightarrow (3,3)$, $Z_1 = 39$

→ Lower Bound (LB) = 39

step 4 : Evaluate Node 2 ($x_2 \geq 4$)

→ Set $x_2 = 4$.

→ from (2): $5x_1 + 9(4) \leq 45 \Rightarrow 5x_1 \leq 9$
$\Rightarrow x_1 = 1.8$

→ from (1): $1.8 + 4 = 5.8 \leq 6$ (satisfied)

NOTES

Date .....

→ Result: (1.8, 4) with $Z_2 = 41$.

→ Since $41 > 39$, this node is still alive. we branch on $x_1 = 1.8$

step 5 : Second branching (Nodes 3 & 4)

→ Branch 3 (Node 3): $x_2 \geq 4$ & $x_1 \leq 1$

→ Branch 4 (Node 4): $x_2 \geq 4$ & $x_1 \geq 2$

step 6 : Evaluate Node 4

→ If $x_1 = 2$ & $x_2 = 4$

→ from (2): $5(2) + 9(4) = 46 \not\leq 45$

→ constraint violation → "Prune" this branch due to "infeasibility".

step 7 : Evaluate Node 3

→ maximize $x_2 \Rightarrow$ set $x_1 = 1$.

→ from (2): $5(1) + 9x_2 \leq 45 \Rightarrow 9x_2 \leq 40 \Rightarrow x_2 = 4.44$

NOTES

Date .....

→ LP obj $Z_3 = 5(1) + 8(4.44) = 40.52$

→ Branching again on x_2 :

if $x_2 = 4 \Rightarrow Z = 5(1) + 8(4) = 37$

$\Rightarrow$ Worse than 39 $\Rightarrow$ Prune

if $x_2 = 5$,

from (2) $\Rightarrow x_1 = 0$ & $5(0) + 9(5) \leq 45$

$0 + 5 \leq 6$

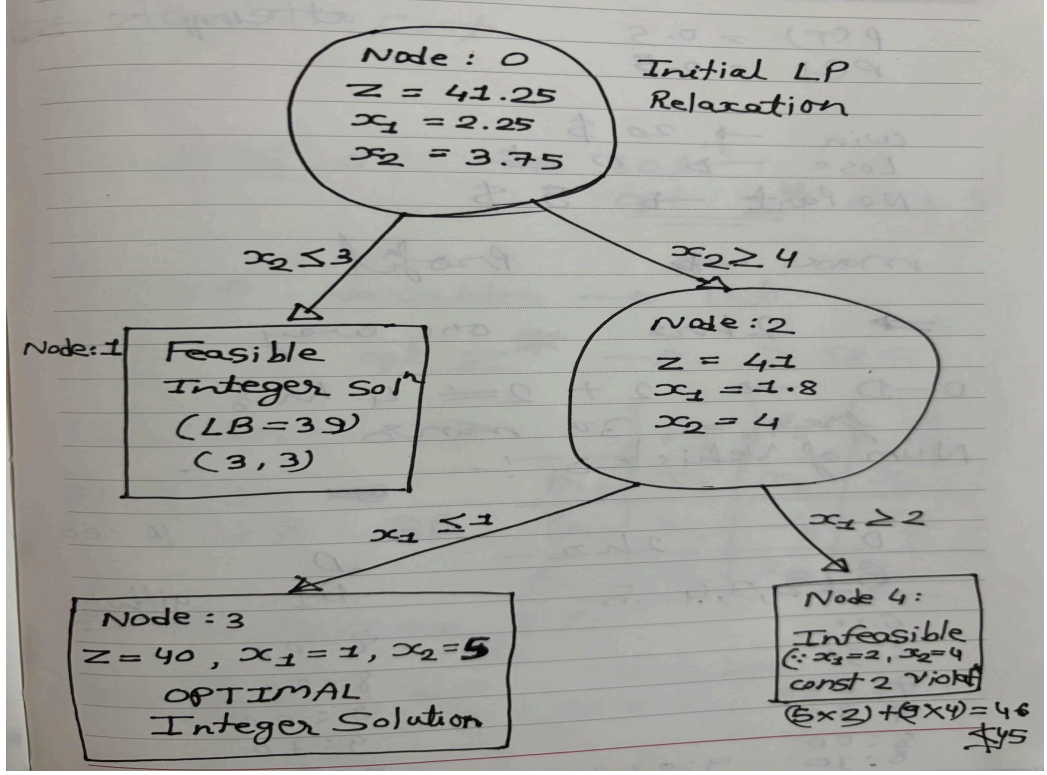
$\therefore$ Integer sol^n found, $Z_4 = 5(0) + 8(5)$

$\boxed{Z = 40}$

$\therefore$ Update LB $= 40$

$\Rightarrow$ Best Integer sol^n : $x_1 = 0$, $x_2 = 5$

optimal objective Value : $Z = 40$

NOTES

Date .....